

Robotics 2

Trajectory Tracking Control

Prof. Alessandro De Luca

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI



SAPIENZA
UNIVERSITÀ DI ROMA

Inverse dynamics control

given the dynamic model of a rigid robot manipulator

$$M(q)\ddot{q} + n(q,\dot{q}) = \tau$$

$$c(q,\dot{q}) + g(q) + \text{friction model}$$

and a twice-differentiable desired trajectory for $t \in [0, T]$

$$q_d(t) \;\rightarrow\; \dot{q}_d(t), \ddot{q}_d(t)$$

applying the feedforward torque in nominal conditions

$$\tau_d = M(q_d)\ddot{q}_d + n(q_d,\dot{q}_d)$$

yields exact reproduction of the desired motion, provided that $q(0) = q_d(0), \dot{q}(0) = \dot{q}_d(0)$ (initial matched state)

In practice ...

many differences from the nominal condition

- initial state is "not matched" to the desired trajectory $q_d(t)$
- uncertain knowledge of robot dynamic parameters $\pi \longleftrightarrow \hat{\pi}$ (link masses, inertias, CoM positions, …) or coefficients $\rho \longleftrightarrow \hat{\rho}$
- unknown value of the carried/lost payload
- unmodeled dynamics (more complex friction phenomena, transmission elasticity, link flexibility, …)
- transient disturbances on actuators (e.g., power losses), possibly also from unexpected contacts or collisions
- approximation or truncation errors on data



require at least the use of feedback information

Introducing feedback

$$\hat{\tau}_d = \widehat{M}(q_d)\ddot{q}_d + \hat{n}(q_d, \dot{q}_d)$$

with $\widehat{M}$, $\hat{n}$ estimates of terms (or coefficients) in the dynamic model

$\hat{\tau}_d$ can be computed offline, e.g., by $\widehat{NE}_\alpha(q_d, \dot{q}_d, \ddot{q}_d)$

feedback is introduced to make a control scheme more robust

different possible implementations depending on the amount of computational load share

- offline ($\Leftrightarrow$ in open loop)
- online ($\Leftrightarrow$ in closed loop)

two-step control design

1. compensation (feedforward) or cancellation (feedback) of nonlinearities
2. synthesis of a linear control law stabilizing the trajectory error to zero

A series of trajectory controllers

(assuming the nominal case: $\widehat{M} = M, \hat{n} = n$)



1. inverse dynamics compensation (FFW) + PD

$$\tau = \hat{\tau}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q})$$

2. inverse dynamics compensation (FFW) + variable PD

$$\tau = \hat{\tau}_d + \widehat{M}(q_d)(K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}))$$

local stabilization of trajectory error $e(t) = q_d(t) - q(t)$

global, with stronger conditions beyond just $K_P > 0$ and $K_D > 0$

3. feedback linearization (FBL) + [PD+FFW] = "computed torque law"

$$\tau = \widehat{M}(q)(\ddot{q}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q})) + \hat{n}(q, \dot{q})$$

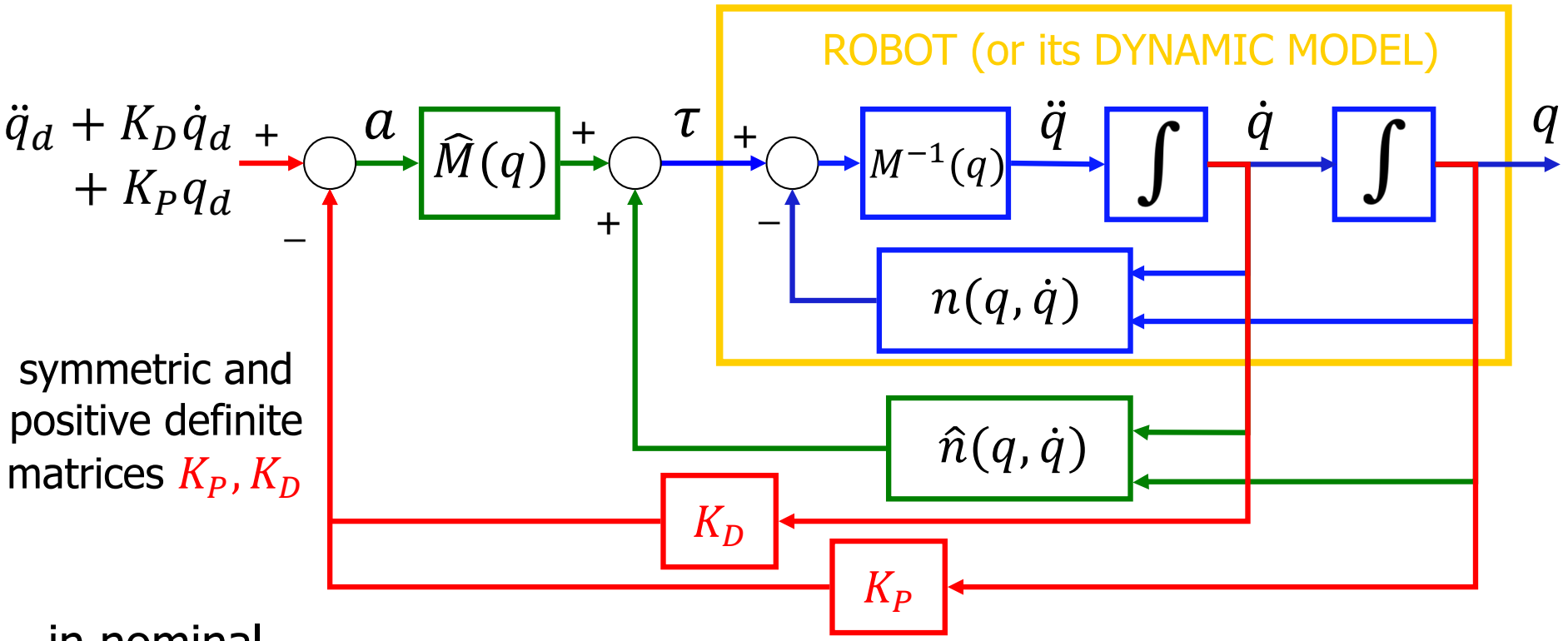
4. feedback linearization (FBL) + [PID+FFW]

$$\tau = \widehat{M}(q)\left(\ddot{q}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) + K_I \int (q_d - q)dt\right) + \hat{n}(q, \dot{q})$$

global stabilization for any $K_P > 0$, $K_D > 0$ (and not too large $K_I > 0$)

more robust to small uncertainties/disturbances, even if more complex to implement in real time

Feedback linearization control



symmetric and positive definite matrices K_P, K_D

in nominal conditions $(\widehat{M} = M, \hat{n} = n)$

$$M(q)\ddot{q} + n(q,\dot{q}) = \tau = M(q)a + n(q,\dot{q}) \quad \Rightarrow \quad \ddot{q} = a$$

nonlinear robot dynamics

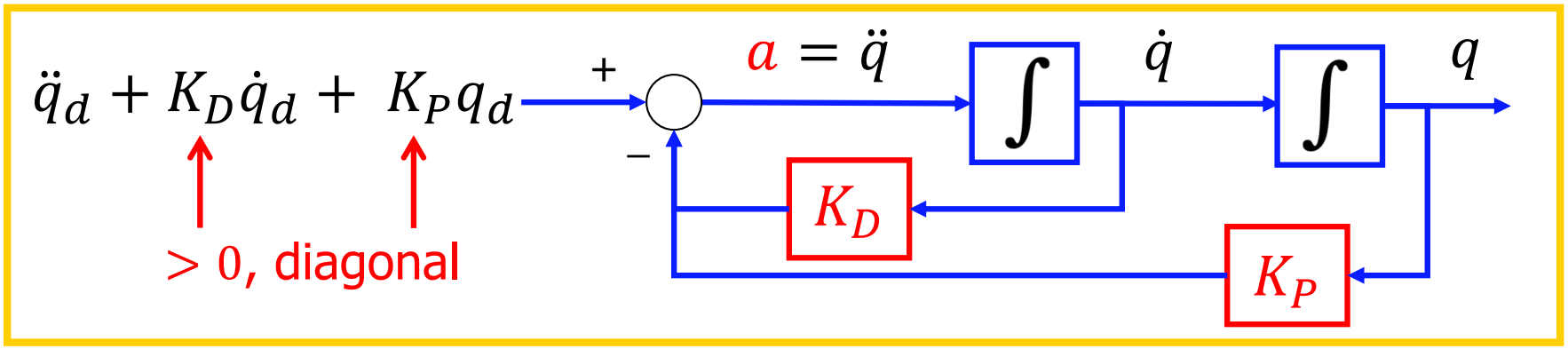
nonlinear control law

linear and decoupled system

global **exponential** stabilization of tracking error

$$a = \ddot{q}_d + K_D(\dot{q}_d - \dot{q}) + K_P(q_d - q)$$

Interpretation in the linear domain



under FBL control, the robot dynamic behavior in the whole state space is

invariant

a unitary mass ($m = 1$), or any desired mass $m_d > 0$, in joint space $\forall(q, \dot{q})$

linear

error transients $e_i = q_{di} - q_i \rightarrow 0$ exponentially, prescribed by K_{Pi}, K_{Di} choice

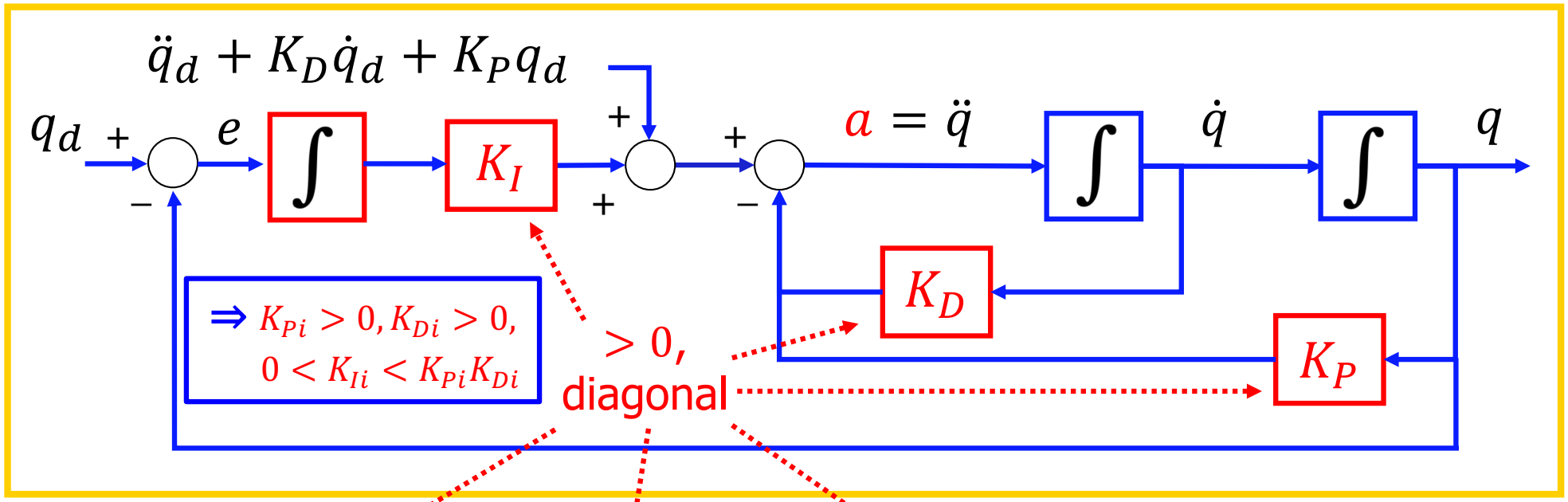
decoupled

each joint coordinate q_i evolves independently from the others, forced by a_i

$$\ddot{e} + K_D\dot{e} + K_P e = 0 \quad \Leftrightarrow \quad \ddot{e}_i + K_{Di}\dot{e}_i + K_{Pi}e_i = 0$$

Addition of an integral term — PID

whiteboard …



$$\ddot{q} = a = \ddot{q}_d + K_D(\dot{q}_d - \dot{q}) + K_P(q_d - q) + K_I \int (q_d - q)\, d\tau \qquad e = q_d - q$$

$$\overset{(1)}{\Rightarrow} \; e_i = q_{di} - q_i \;\; (i = 1, .., n) \qquad \overset{(2)}{\Rightarrow} \; \ddot{e}_i + K_{Di}\dot{e}_i + K_{Pi}e_i + K_{Ii} \int e_i d\tau = 0$$

$$\mathcal{L}[e_i(t)] \overset{(3)}{\Rightarrow} \left(s^2 + K_{Di}s + K_{Pi} + K_{Ii}\frac{1}{s}\right) e_i(s) = 0$$

$$s \times \overset{(4)}{\Rightarrow} (s^3 + K_{Di}s^2 + K_{Pi}s + K_{Ii}) e_i(s) = 0 \overset{(5)}{\Rightarrow}$$

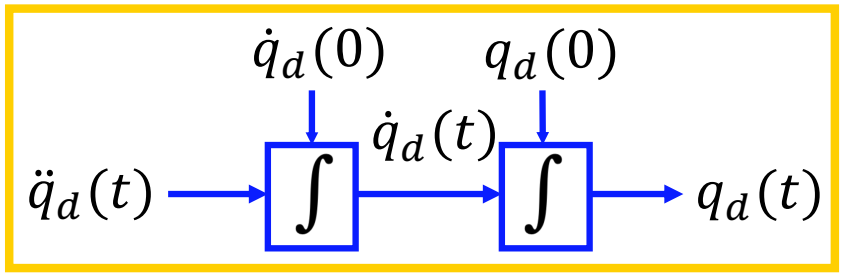
3	1	K_{Pi}
2	K_{Di}	K_{Ii}
1	$(K_{Di}K_{Pi} - K_{Ii})/K_{Di}$	
0	K_{Ii}	

$\Rightarrow$ **(6)** nec & suff conditions exponential stability by Routh criterion

Remarks

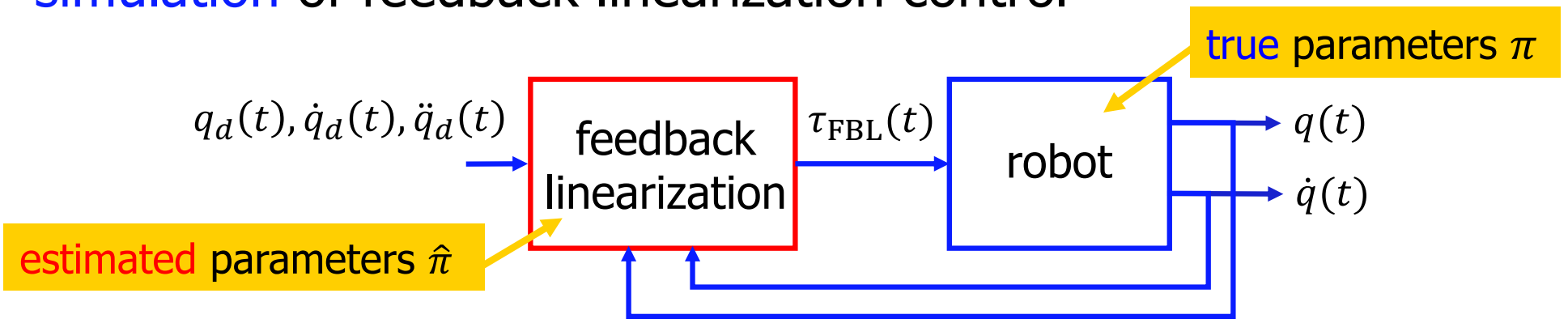
- desired joint trajectory can be generated from Cartesian data

$\ddot{y}_d(t), \dot{y}_d(0), y_d(0)$



$$q_d(0) = k^{-1}\big(y_d(0)\big)$$
$$\dot{q}_d(0) = J^{-1}\big(q_d(0)\big)\dot{y}_d(0)$$
$$\ddot{q}_d(t) = J^{-1}(q_d)\big(\ddot{y}_d(t) - \dot{J}(q_d)\dot{q}_d\big)$$

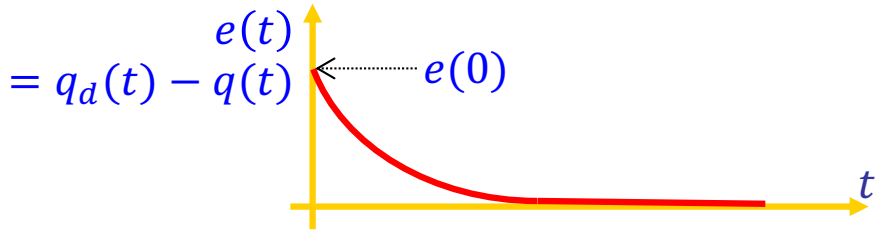
- real-time computation by Newton-Euler $u_{FBL} = \widehat{NE}_{\alpha}(q, \dot{q}, a)$
- simulation of feedback linearization control



Hint: no sense in simulating the FBL control law in the ideal case ($\hat{\pi} = \pi$) since the robot behavior will be the same as that of n linear, decoupled and stabilized double integrators!

Further comments

- choice of the diagonal elements of K_P, K_D (and K_I, if present)
  - shaping the error transients, with an eye also to motor saturations...



$K_{Di}^2 < 2K_{pi}$ underdamped (oscillating)

$K_{Di}^2 = 2K_{pi}$ critically damped (Butterworth)

$K_{Di}^2 = 4K_{pi}$ no overshoot

$K_{Di}^2 > 4K_{pi}$ overdamped (too slow)

- parametric identification
  - to be done in advance, using the property of linearity in the dynamic coefficients of the robot dynamic model
- choice of the sampling time of a digital implementation
  - compromise between computational time and tracking accuracy, typically $T_c = 0.5 \div 10$ ms
- exact linearization by (state) feedback is a general technique of nonlinear control theory
  - can be used for robots with elastic joints, wheeled mobile robots, ...
  - non-robotics applications: satellites, induction motors, helicopters, ...

Another example of feedback linearization design



- dynamic model of robots with elastic joints
  - q = link position
  - θ = motor position (after reduction gears)

  } $2n$ generalized coordinates (q, θ)
  - M_m = diagonal matrix (> 0) of inertia of the (balanced) motors
  - K = diagonal matrix (> 0) of (finite) stiffness of the joints

$4n$ state variables $x = (q, \theta, \dot{q}, \dot{\theta})$

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) + K(q - \theta) = 0 \qquad (1)$$

$$M_m\ddot{\theta} + K(\theta - q) = \tau \qquad (2)$$

- there is a control law that achieves exact linearization via feedback

$$\tau = \alpha\left(q, \theta, \dot{q}, \dot{\theta}\right) + \beta\left(q, \theta, \dot{q}, \dot{\theta}\right) a$$

$$\Rightarrow \quad \frac{d^4 q_i}{dt^4} = a_i, \quad i = 1, .., n \quad \Leftrightarrow$$

linear and decoupled system of n chains of 4 integrators to be stabilized by linear control design

Hint: differentiate (1) w.r.t. time until motor acceleration $\ddot{\theta}$ appears; substitute this from (2); choose τ to cancel all nonlinearities; rewrite $\ddot{q}$ and $\dddot{q}$ as functions of x …

Alternative global trajectory controller

$$\tau = M(q)\ddot{q}_d + C(q,\dot{q})\dot{q}_d + g(q) + B_v\dot{q}_d + K_P e + K_D \dot{e}$$

special factorization such that $\dot{M} - 2C$ is skew-symmetric

e.g., viscous friction

symmetric and positive definite matrices

- global asymptotic stability of $(e, \dot{e}) = (0,0)$ (trajectory tracking)
- proven by Lyapunov + Barbalat (time-varying system) + LaSalle
- does not produce a complete cancellation of nonlinearities
  - variables $\dot{q}$ and $\ddot{q}$ that appear linearly in the model are evaluated on the desired trajectory $q_d(t)$
- does not induce a linear and decoupled behavior of the trajectory error $e(t) = q_d(t) - q(t)$ in the closed-loop system
- however, it lends itself more easily to an adaptive version
- computation by $4\times$ standard or $1\times$ modified NE algorithm

Analysis of asymptotic stability
of the trajectory error - 1



$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + B_v\dot{q} = \tau$$ robot dynamics (including friction)

control law $\tau = M(q)\ddot{q}_d + C(q,\dot{q})\dot{q}_d + g(q) + B_v\dot{q}_d + K_P e + K_D\dot{e}$

- Lyapunov candidate and its time derivative (with $e = q_d - q$)

$$V = \frac{1}{2}\dot{e}^T M(q)\dot{e} + \frac{1}{2}e^T K_P e \geq 0 \;\Rightarrow\; \dot{V} = \frac{1}{2}\dot{e}^T\dot{M}(q)\dot{e} + \dot{e}^T M(q)\ddot{e} + e^T K_P\dot{e}$$

- the closed-loop system equations yield

$$M(q)\ddot{e} = -C(q,\dot{q})\dot{e} - (K_D + B_v)\dot{e} - K_P e$$

- substituting and using the property of skew-symmetry of $\dot{M} - 2C$

$$\dot{V} = -\dot{e}^T(K_D + B_v)\dot{e} \leq 0 \qquad \dot{V} = 0 \;\Leftrightarrow\; \dot{e} = 0$$

- since the system is time-varying (due to $q_d(t)$), direct application of LaSalle theorem is NOT allowed ⇒ use Barbalat lemma …

$$q = q_d(t) - e, \dot{q} = \dot{q}_d(t) - \dot{e} \;\Rightarrow\; V = V(e,\dot{e},t) = V(x,t)$$

error state x

$\ddot{e}$ (or $\ddot{q}$) should be written **only** in terms of the state, thus either $(q,\dot{q})$ or $(e,\dot{e})$, and **not** using all quantities $(q,\dot{q},e,\dot{e})$; by doing so, a dependence on time through $q_d(t)$ would appear

⇒ go to slide 10 in block 8

Analysis of asymptotic stability
of the trajectory error - 2



- since i) V is lower bounded and ii) $\dot{V} \leq 0$, we have to check only condition iii) in order to apply Barbalat lemma

$$\ddot{V} = -2\dot{e}^T(K_D + B_v)\ddot{e} \quad \text{... is this bounded?}$$

- from i) + ii), V is bounded $\Rightarrow$ e and $\dot{e}$ are bounded
- assume that the desired trajectory has bounded velocity $\dot{q}_d$

$\Rightarrow$ $\dot{q}$ is bounded

- using the following two properties of dynamic model terms

$$0 < \alpha_m \leq \|M^{-1}(q)\| \leq \alpha_M < \infty \qquad \|C(q,\dot{q})\| \leq \alpha_C\|\dot{q}\|$$

then also $\ddot{e}$ will be bounded (in norm) since

$$\ddot{e} = -M^{-1}(q)[C(q,\dot{q})\dot{e} + K_P e + (K_D + B_v)\dot{e}]$$

bounded in norm by α_M ; bounded in norm by $\alpha_C\|\dot{q}\|$; bounded ; bounded ; bounded ; bounded

$\Rightarrow \lim_{t\to\infty} \dot{V}(t) = 0$

Analysis of asymptotic stability
of the trajectory error – end of proof



- we can conclude by proceeding as in LaSalle theorem

$$\dot{V} = 0 \quad \Leftrightarrow \quad \dot{e} = 0$$

- the closed-loop dynamics in this situation is

$$M(q)\ddot{e} = -K_P e$$

$$\Longrightarrow \quad \ddot{e} = 0 \quad \Leftrightarrow \quad e = 0 \quad \Rightarrow \quad (e, \dot{e}) = (0, 0)$$

is the largest invariant set in $\dot{V} = 0$

$\Rightarrow$ (global) asymptotic tracking will be achieved

Regulation as a special case



- what happens to the control laws designed for trajectory tracking when q_d is constant? are there simplifications?
- feedback linearization

$$\tau = M(q)(K_P(q_d - q) - K_D\dot{q}) + c(q,\dot{q}) + g(q)$$

  - no special simplifications
  - however, this is a solution to the regulation problem with exponential stability (and decoupled transients at each joint!)
- alternative global controller

$$\tau = K_P(q_d - q) - K_D\dot{q} + g(q)$$

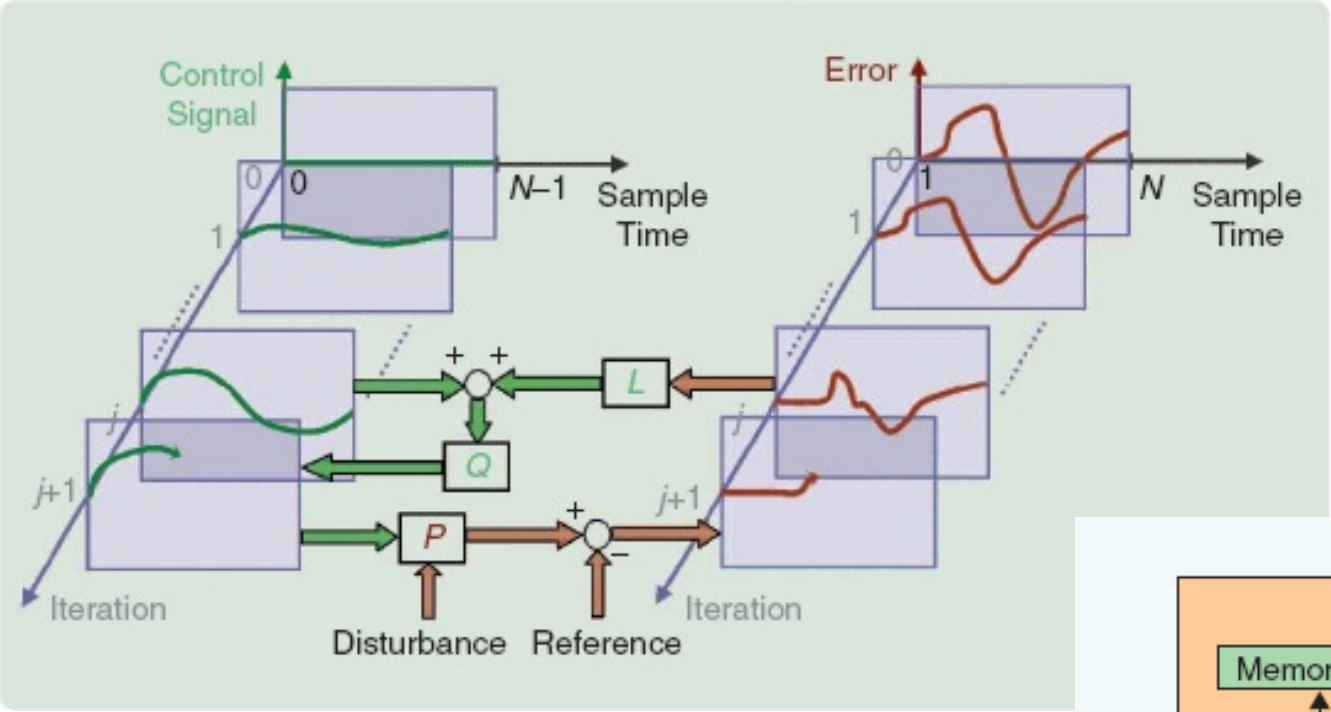
  - we recover the simpler PD + gravity cancellation control law!!

Trajectory execution without a model



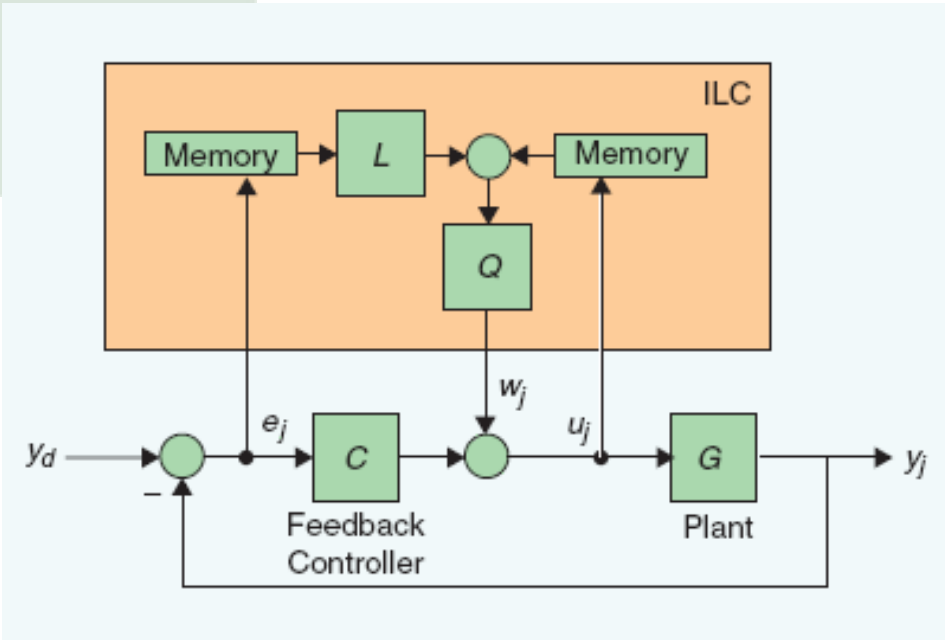
- is it possible to accurately reproduce a desired smooth joint-space reference trajectory with reduced or no information on the robot dynamic model?
- this is feasible (and even rather simple) in case of repetitive motion tasks over a finite interval of time
  - trials are performed iteratively, storing trajectory error and control effort of the current execution [k-th iteration] and processing it offline before the next trial [$(k+1)$-iteration] starts
  - the robot should be reinitialized in the same initial state at the beginning of each trial (typically, with $\dot{q} = 0$)
  - the control law is made of a model-independent part (often, a decentralized PD) + a time-varying feedforward which is updated before every trial
- this scheme is called iterative (trajectory) learning control

Iterative trajectory learning



- learning-based method for high-performance trajectory tracking control
- representation of control and error signals in 2D
  - along execution time
  - along trial iterations
- ILC = Iterative Learning Control

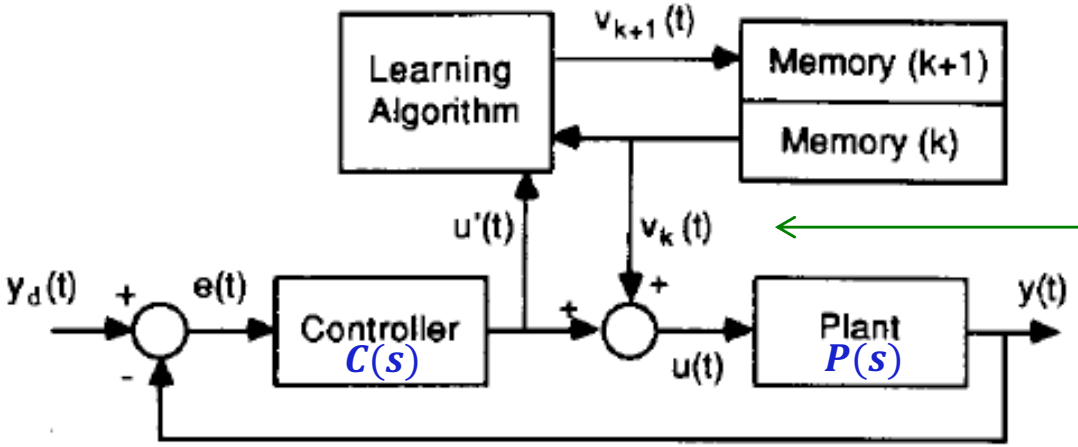
- feedforward is updated from iteration to iteration
- at the end of each trial, error and control effort are stored and their (filtered) combination is used to generate the feedforward for the next iteration
- many implementations using different system signals



D.A. Bristow, M. Tharayil, A.G. Alleyne, "A Survey of Iterative Learning Control," *IEEE Control Systems Mag.*, 26(3), 96-114, 2006 (available in course material)

Scheme of iterative trajectory learning

- control design can be illustrated on a SISO linear system in the Laplace domain



"plug-in" signal: $v_1(t) \equiv 0$ at first ($k = 1$) iteration

$$W(s) = \frac{y(s)}{y_d(s)} = \frac{P(s)C(s)}{1 + P(s)C(s)}$$

closed-loop system without learning ($C(s)$ is, e.g., a PD control law)

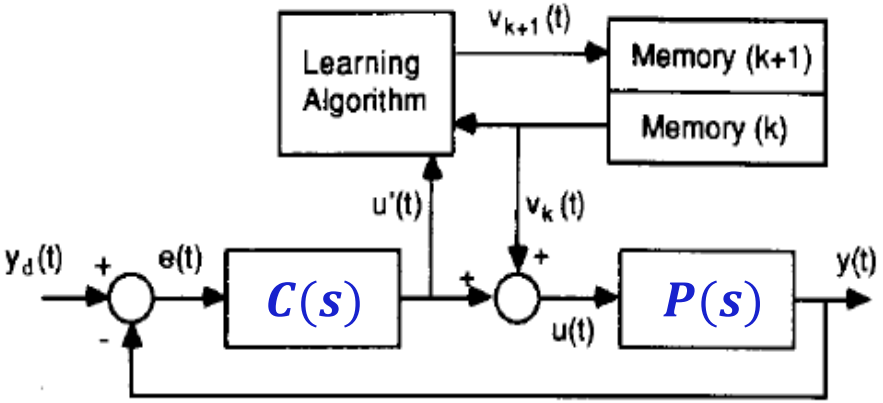
$$u_k(s) = u'_k(s) + v_k(s) = C(s)e_k(s) + v_k(s)$$ control law at iteration k

$$y_k(s) = W(s)y_d(s) + \frac{P(s)}{1 + P(s)C(s)}v_k(s)$$ system output at iteration k

Background math on feedback loops

whiteboard…

- algebraic manipulations on block diagram signals in the Laplace domain:
  $x(s) = \mathcal{L}[x(t)]$, $x = \{y_d, y, u', v, e\} \Rightarrow \{y_d, y_k, u'_k, v_k, e_k\}$, with transfer functions



$$y(s) = P(s)u(s) = P(s)\big(v(s) + u'(s)\big)$$
$$= P(s)v(s) + P(s)C(s)e(s)$$
$$= P(s)v(s) + P(s)C(s)(y_d(s) - y(s))$$

$$\Rightarrow \; (1 + P(s)C(s))\, y(s) =$$
$$= P(s)v(s) + P(s)C(s)y_d(s)$$

$$\Rightarrow \; y(s) = \frac{P(s)C(s)}{1 + P(s)C(s)}\, y_d(s) + \frac{P(s)}{1 + P(s)C(s)}\, v(s) = W(s)y_d(s) + W_v(s)v(s)$$

- feedback control law at iteration k

$$u'_k(s) = C(s)\big(y_d(s) - y_k(s)\big) = C(s)y_d(s) - P(s)C(s)\big(v_k(s) + u'_k(s)\big)$$

$$\Rightarrow \; u'_k(s) = \frac{C(s)}{1 + P(s)C(s)}\, y_d(s) - \frac{P(s)C(s)}{1 + P(s)C(s)}\, v_k(s) = W_c(s)y_d(s) - W(s)v_k(s)$$

- error at iteration k

$$e_k(s) = y_d(s) - y_k(s) = y_d(s) - \big(W(s)y_d(s) + W_v(s)v_k(s)\big) = \big(1 - W(s)\big)y_d(s) - W_v(s)v_k(s)$$

$$W_e(s) = 1/(1 + P(s)C(s))$$

Learning update law

- the update of the feedforward term is designed as

$$v_{k+1}(s) = \alpha(s)u'_k(s) + \beta(s)v_k(s)$$

with α and β suitable filters (even non-causal, of the FIR type)

recursive expression of feedforward term

$$v_{k+1}(s) = \frac{\alpha(s)C(s)}{1 + P(s)C(s)} y_d(s) + (\beta(s) - \alpha(s)W(s))v_k(s)$$

recursive expression of error $e = y_d - y$

$$e_{k+1}(s) = \frac{1 - \beta(s)}{1 + P(s)C(s)} y_d(s) + \big(\beta(s) - \alpha(s)W(s)\big)e_k(s)$$

- if a contraction condition can be enforced

$$|\beta(s) - \alpha(s)W(s)| < 1$$

(for all $s = j\omega$ frequencies such that ...)

then convergence is obtained for $k \to \infty$

$$v_\infty(s) = \frac{y_d(s)}{P(s)} \frac{\alpha(s)W(s)}{1 - \beta(s) + \alpha(s)W(s)} \quad e_\infty(s) = \frac{y_d(s)}{1 + P(s)C(s)} \frac{1 - \beta(s)}{1 - \beta(s) + \alpha(s)W(s)}$$

Proof of recursive updates

whiteboard …



- recursive expression for the feedforward v_k

$$v_{k+1}(s) = \alpha(s)u'_k(s) + \beta(s)v_k(s) = \alpha(s)C(s)e_k(s) + \beta(s)v_k(s)$$
$$= \alpha(s)C(s)\left(W_e(s)y_d(s) - W_v(s)v_k(s)\right) + \beta(s)v_k(s)$$
$$= \frac{\alpha(s)C(s)}{1 + P(s)C(s)}\, y_d(s) + (\beta(s) - \alpha(s)W(s))\, v_k(s)$$

- recursive expression for the error e_k

$$e_k(s) = y_d(s) - y_k(s) = y_d(s) - P(s)(v_k(s) + u'_k(s))$$
$$\Rightarrow\ v_k(s) = \frac{1}{P(s)}\left(y_d(s) - e_k(s)\right) - u'_k(s)$$

$$y_{k+1}(s) = P(s)\left(v_{k+1}(s) + u'_{k+1}(s)\right) = P(s)\left(\alpha(s)u'_k(s) + \beta(s)v_k(s) + u'_{k+1}(s)\right)$$
$$= P(s)\left(\alpha(s)C(s)e_k(s) + \beta(s)\frac{1}{P(s)}\left(y_d(s) - e_k(s)\right) - \beta(s)C(s)e_k(s) + C(s)e_{k+1}(s)\right)$$

$$e_{k+1}(s) = y_d(s) - y_{k+1}(s)$$
$$= (1 - \beta(s))\, y_d(s) - \left(\left(\alpha(s) - \beta(s)\right)P(s)C(s) - \beta(s)\right)e_k(s) - P(s)C(s)e_{k+1}(s)$$
$$\Rightarrow\ e_{k+1}(s) = \frac{1 - \beta(s)}{1 + P(s)C(s)}\, y_d(s) + \left(\beta(s) - \alpha(s)W(s)\right)e_k(s)$$

Proof of convergence

whiteboard …



from recursive expressions

$$v_{k+1}(s) = \frac{\alpha(s)C(s)}{1+P(s)C(s)} y_d(s) + (\beta(s) - \alpha(s)W(s))\, v_k(s)$$

$$e_{k+1}(s) = \frac{1-\beta(s)}{1+P(s)C(s)} y_d(s) + \big(\beta(s) - \alpha(s)W(s)\big)\, e_k(s)$$

compute variations from k to $k+1$ (repetitive term in trajectory y_d vanishes!)

$$\Delta v_{k+1}(s) = v_{k+1}(s) - v_k(s) = (\beta(s) - \alpha(s)W(s))\, \Delta v_k(s)$$

$$\Delta e_{k+1}(s) = e_{k+1}(s) - e_k(s) \; = \big(\beta(s) - \alpha(s)W(s)\big)\, \Delta e_k(s)$$

by contraction mapping condition $|\beta(s) - \alpha(s)W(s)| < 1 \Rightarrow \{v_k\} \to v_\infty$, $\{e_k\} \to e_\infty$

$$v_\infty(s) = \frac{\alpha(s)C(s)}{1+P(s)C(s)} y_d(s) \; + (\beta(s) - \alpha(s)W(s))\, v_\infty(s)$$

$$e_\infty(s) = \frac{1-\beta(s)}{1+P(s)C(s)} y_d(s) + \big(\beta(s) - \alpha(s)W(s)\big)\, e_\infty(s)$$

$$\Rightarrow \quad v_\infty(s) = \frac{y_d(s)}{P(s)} \frac{\alpha(s)W(s)}{1-\beta(s)+\alpha(s)W(s)} \qquad e_\infty(s) = \frac{y_d(s)}{1+P(s)C(s)} \frac{1-\beta(s)}{1-\beta(s)+\alpha(s)W(s)}$$

Comments on convergence

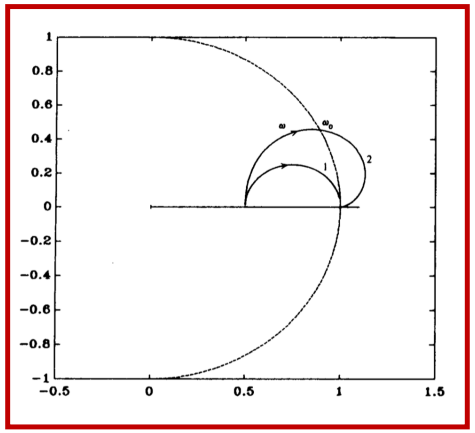
- if the choice $\beta = 1$ allows to satisfy the contraction condition, then convergence to zero tracking error is obtained

$$e_\infty(s) = 0$$

and the inverse dynamics command has been learned

$$v_\infty(s) = \frac{y_d(s)}{P(s)}$$

- for $\alpha(s) = 1/W(s)$ convergence would be in 1 iteration only!
- the use of filter $\beta(s) \neq 1$ allows to obtain convergence (but with residual tracking error) even in presence of unmodeled high-frequency dynamics
- the two filters can be designed from very poor information on system dynamics, using classic tools (e.g., Nyquist plots)



Application to robots

- for n-dof manipulators modeled as

$$(M_m + M(q))\ddot{q} + (B_v + C(q,\dot{q}))\dot{q} + g(q) = \tau$$

we choose as (initial = pre-learning) control law

$$\tau = K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) + \hat{g}(q)$$

this is u' in the block diagram of slide #20

and design the learning filters (at each joint) using the simple linear approximation

$$W_i(s) = \frac{q_i(s)}{q_{di}(s)} = \frac{K_{Di}s + K_{Pi}}{\widehat{M}_{mi}s^2 + \left(\hat{B}_{vi} + K_{Di}\right)s + K_{Pi}} \qquad i = 1, \cdots, n$$

- initialization of feedforward uses the best estimates

$$v_1 = \left(\widehat{M}_m + \widehat{M}(q_d)\right)\ddot{q}_d + \left(\hat{B}_v + \hat{C}(q_d,\dot{q}_d)\right)\dot{q}_d + \hat{g}(q_d)$$

or simply $v_1 = 0$ (in the worst case) at the first trial $k = 1$

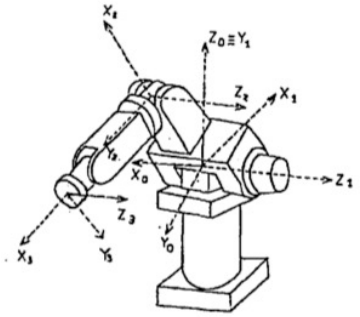
Experimental set-up

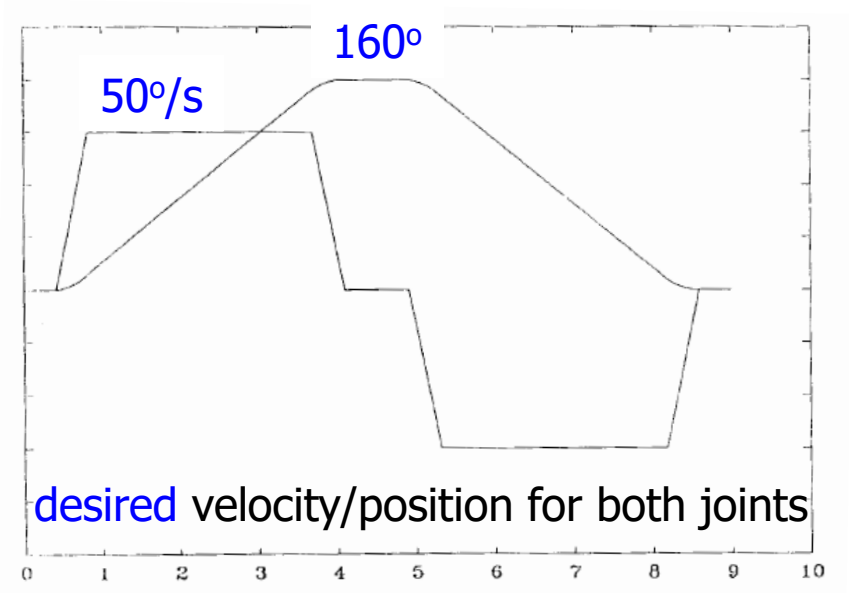
- joints 2 and 3 of 6R MIMO CRF robot prototype @DIS

≈ 90% gravity balanced through springs

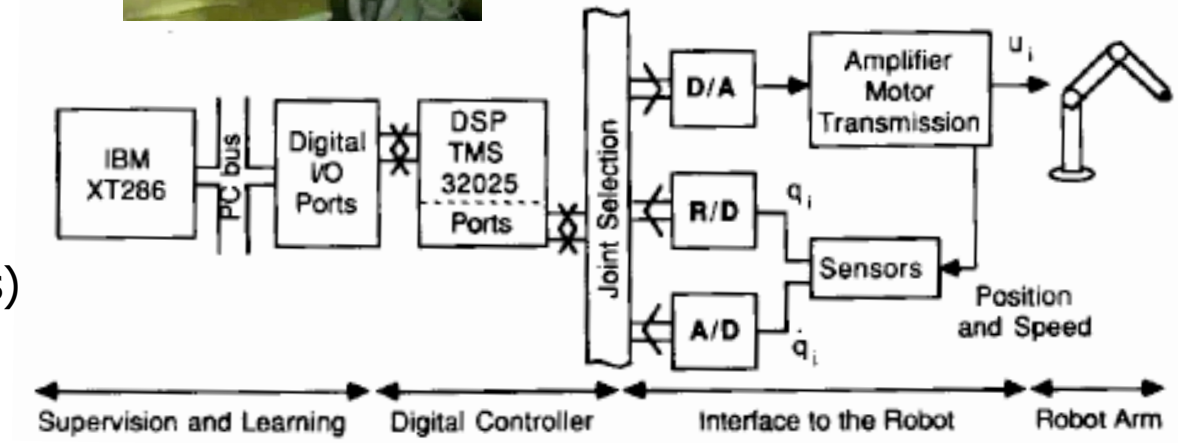
high level of friction

Harmonic Drives transmissions with ratio 160:1





DSP T_c = 400μs
D/A = 12 bit
R/D = 16 bit/2π
A/D = 11 bit/(rad/s)



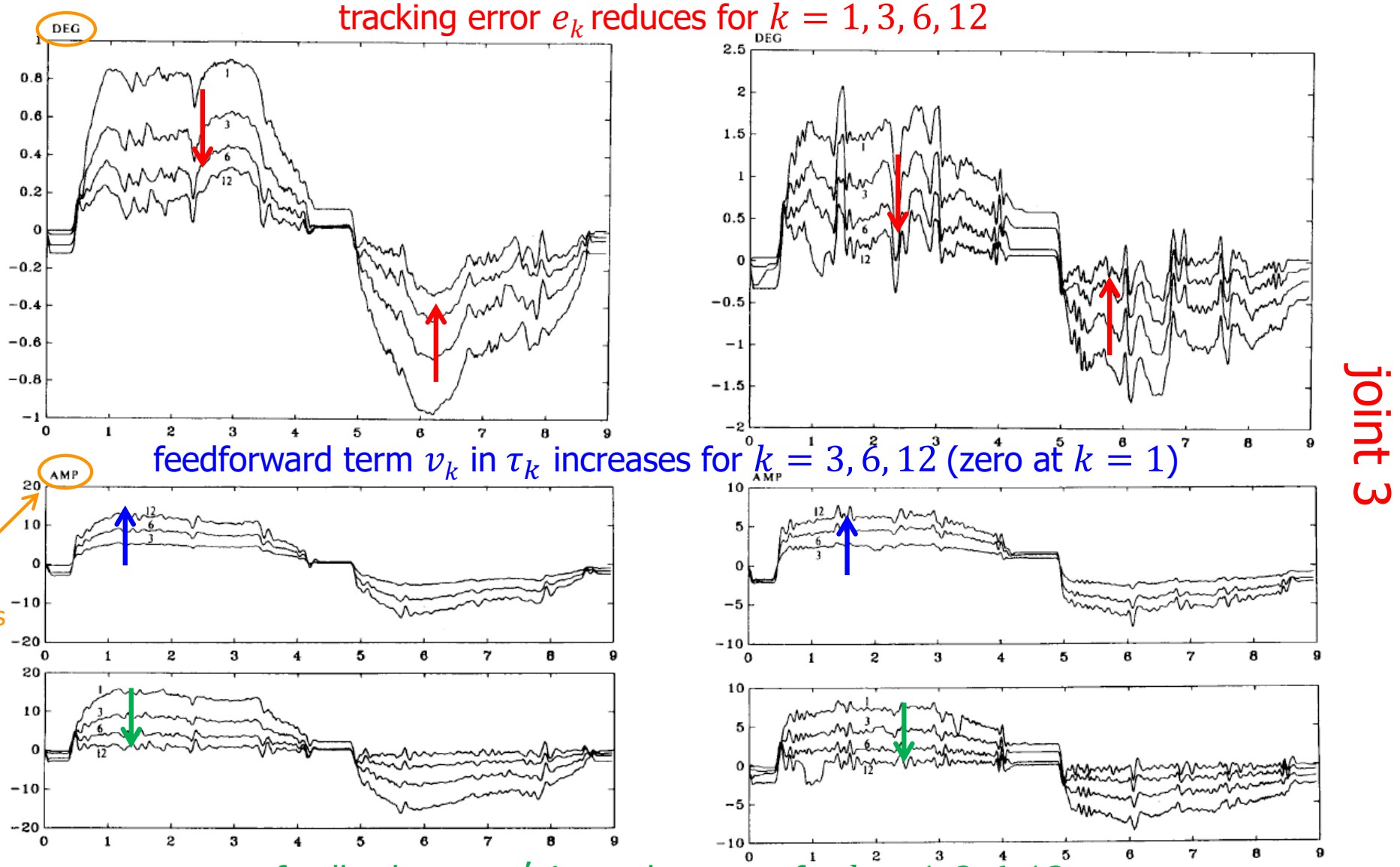
DC motors with current amplifiers

resolvers and tachometers

De Luca, Paesano, Ulivi: IEEE Trans Ind Elect, 1992

Experimental results

tracking error e_k reduces for $k = 1, 3, 6, 12$



feedforward term v_k in τ_k increases for $k = 3, 6, 12$ (zero at $k = 1$)

feedback term u'_k in τ_k decreases for $k = 1, 3, 6, 12$

Online learning control



- re-visitation of the learning idea so to acquire the missing dynamic information in model-based trajectory control
- online learning approach
  - the robot already improves its tracking performance while executing the task in feedback mode
- uses only position measurements from encoders
  - no need of joint torque sensors
- machine learning techniques used for
  - data collection and organization
  - regressor construction for estimating model perturbations
- fast convergence
  - starting with a reasonably good robot model
- extensions to underactuated robots or with flexible components

Control with approximate FBL



- dynamic model, its nominal part and (unstructured) uncertainty

$$M(q)\ddot{q} + n(q,\dot{q}) = \tau \qquad M = \hat{M} + \Delta M \quad n = \hat{n} + \Delta n$$

- model-based (approximate) feedback linearization

$$\tau_{FBL} = \hat{M}(q)a + \hat{n}(q,\dot{q})$$

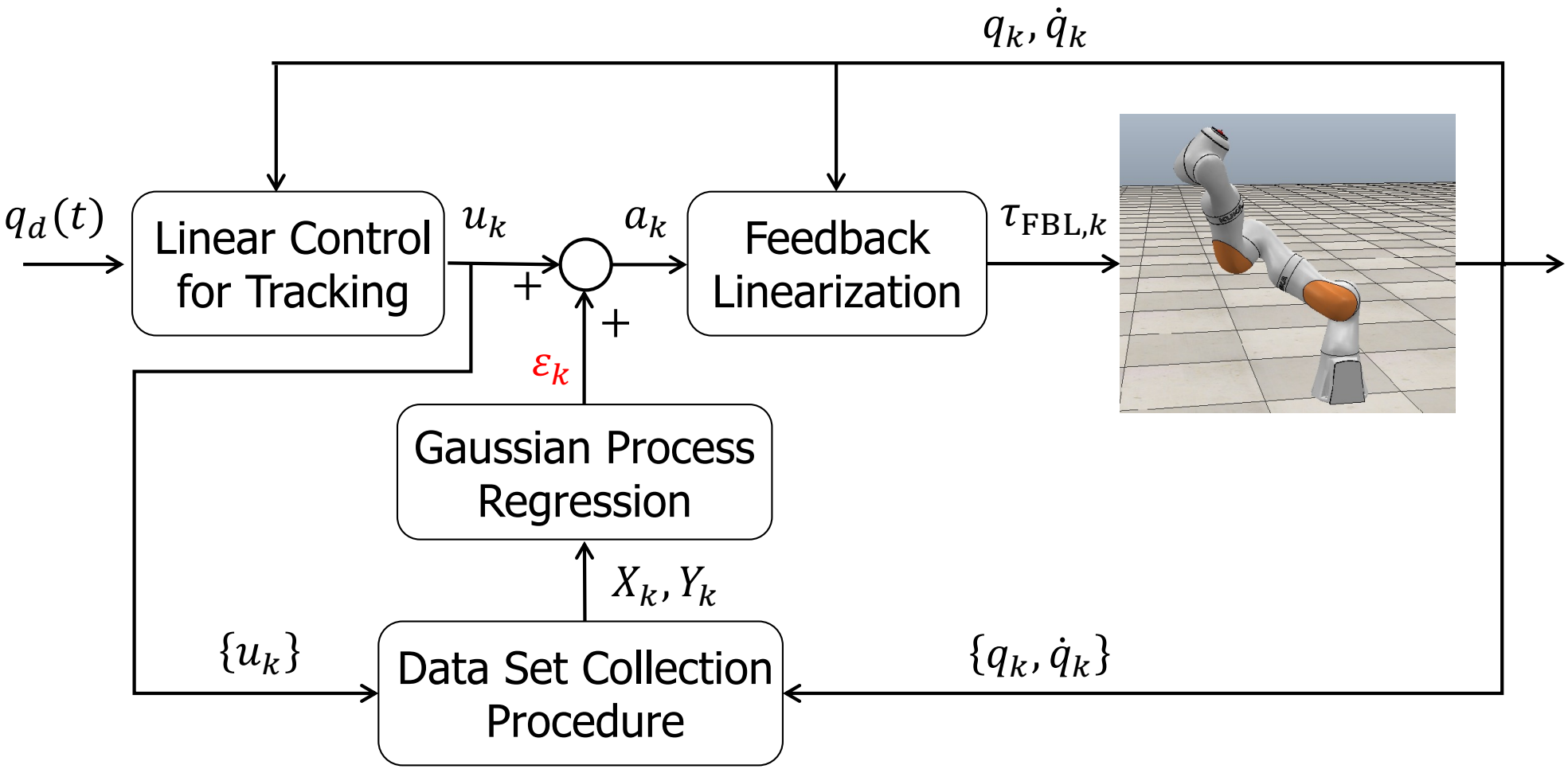
- resulting closed-loop dynamics with perturbation

$$\ddot{q} = a + \delta(q,\dot{q},a) \quad \leftarrow \quad \delta = \left(M^{-1}\hat{M} - I\right)a + M^{-1}(\hat{n} - n)$$

- control law for tracking $q_d(t)$ is completed by using (at $t = t_k$) a linear design (PD with feedforward) and a learning regressor ε_k

$$a = a_k = u_k + \varepsilon_k$$
$$= \ddot{q}_{d,k} + K_P(q_{d,k} - q_k) + K_D(\dot{q}_{d,k} - \dot{q}_k) + \varepsilon_k$$

Online learning scheme



Online regressor

- Gaussian Process (GP) regression to estimate the perturbation δ
  - from input-output observations that are noisy, with $\omega \sim \mathcal{N}(0, \Sigma_\omega)$, the generated data points at the k-th control step are

$$X_k = (q_k, \dot{q}_k, u_k) \qquad Y_k = \ddot{q}_k - u_k$$

  - assuming the ensemble of n_d observations with a joint Gaussian distribution

$$\begin{pmatrix} Y_{1:n_d-1} \\ Y_{n_d} \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K & k \\ k^T & \kappa(X_{n_d}, X_{n_d}) \end{pmatrix}\right)$$

a Kernel to be chosen

  - the predictive distribution that approximates $\delta(\hat{X})$ for a generic query $\hat{X}$ is

$$\varepsilon(\hat{X}) \sim \mathcal{N}\left(\mu(\hat{X}), \sigma^2(\hat{X})\right)$$

with

$$\mu(\hat{X}) = k^T(\hat{X})(K + \Sigma_\omega)^{-1} Y$$

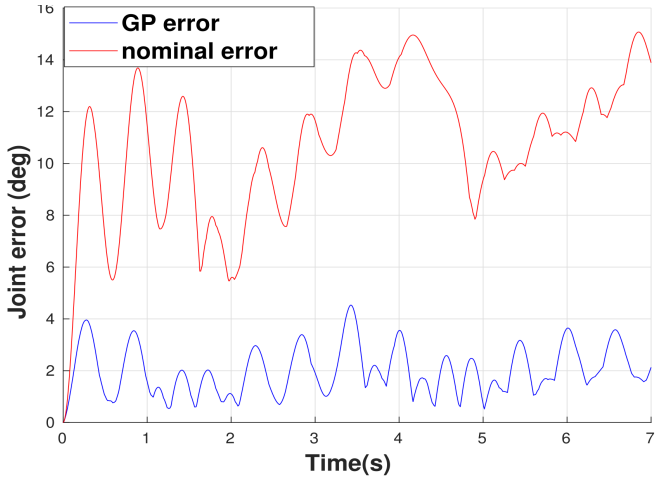
$$\sigma^2(\hat{X}) = \kappa(\hat{X}, \hat{X}) - k^T(\hat{X})(K + \Sigma_\omega)^{-1} k(\hat{X})$$

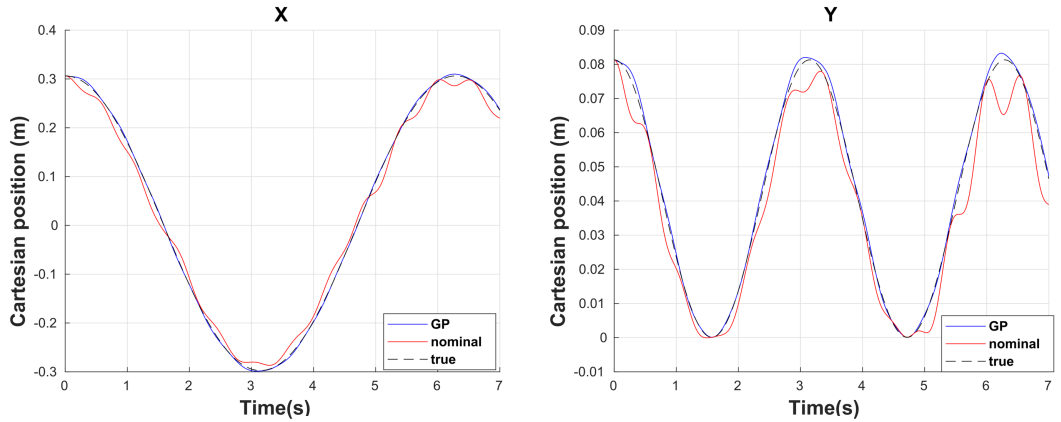
$$\Rightarrow \varepsilon_k = \varepsilon(X_k)$$

Simulation results

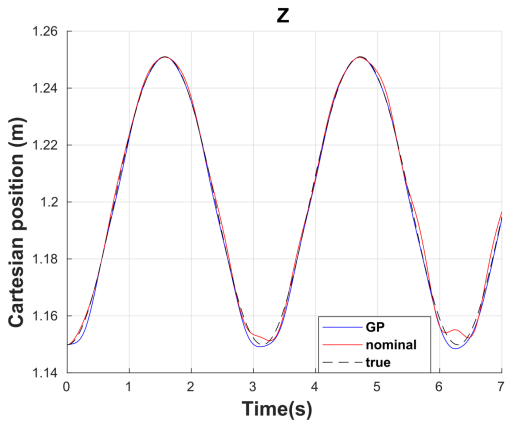
- Kuka LWR iiwa, 7-dof robot
- model perturbations: dynamic parameters with $\pm$ 20% variation, uncompensated joint friction
- 7 separate GPs (one for each joint), each with 21 inputs at every $t = t_k$
- sinusoidal trajectories for each joint

norm of the joint errors





position components in the Cartesian space



... at the first and only iteration!

Simulation results



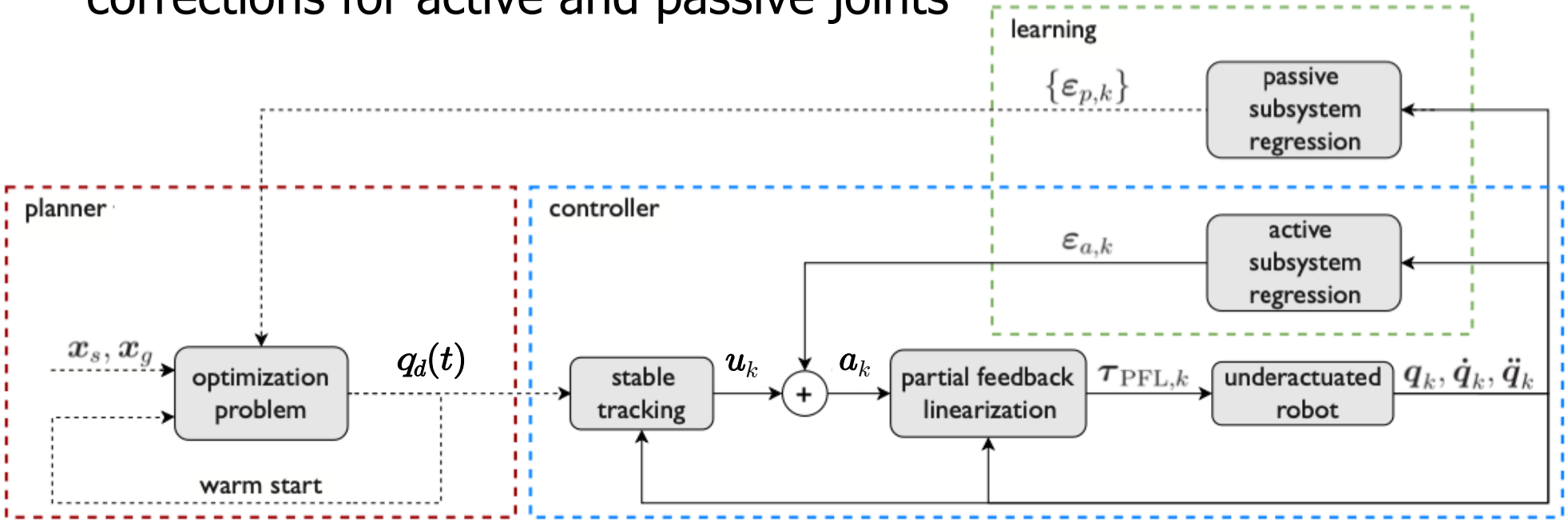
video
(slowed down)

Proc. of Machine Learning Research, vol. 100 (2020)

Extension to underactuated robots

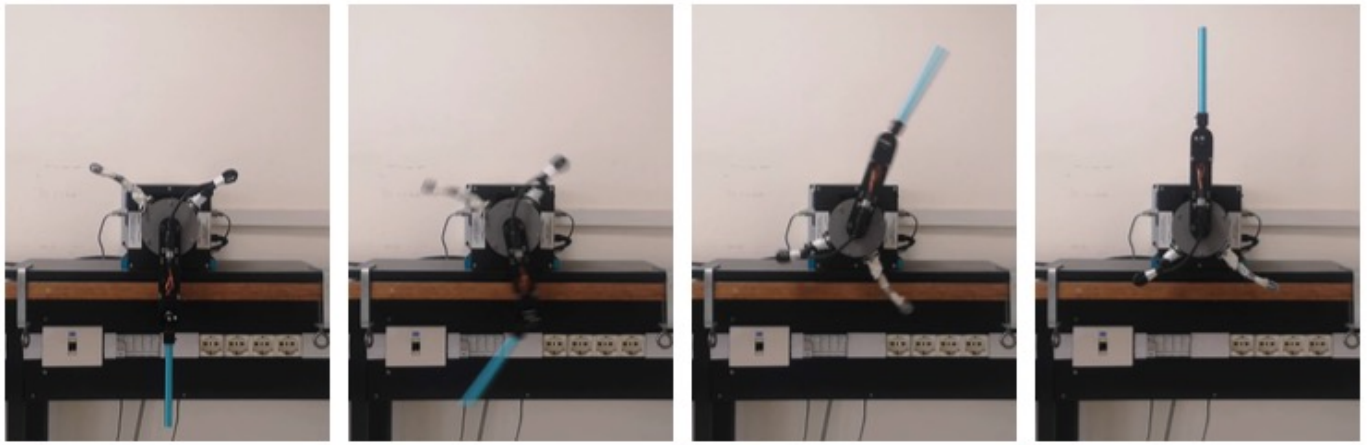
$$\begin{pmatrix} M_{aa}(q) & M_{ap}(q) \\ M_{ap}^T(q) & M_{pp}(q) \end{pmatrix} \begin{pmatrix} \ddot{q}_a \\ \ddot{q}_p \end{pmatrix} + \begin{pmatrix} n_a(q,\dot{q}) \\ n_p(q,\dot{q}) \end{pmatrix} = \begin{pmatrix} \tau \\ 0 \end{pmatrix}$$

- planner optimizes motion of passive joints (at every iteration)
- controller for active joints with partial feedback linearization
- two regressors (on/off-line) for learning the required acceleration corrections for active and passive joints

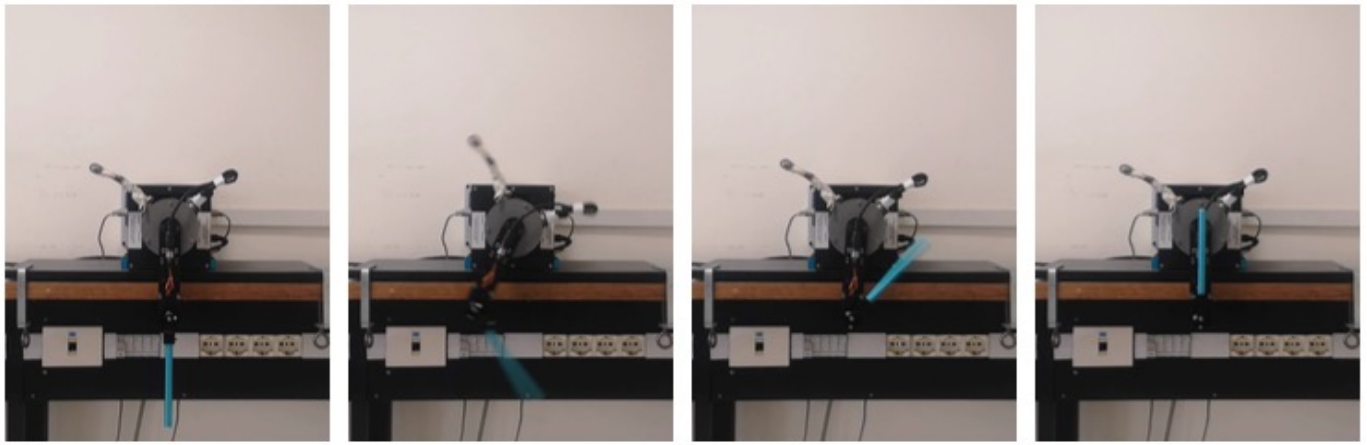


Experiments on the Pendubot

- Pendubot, 2-dof robot with passive second joint
- swing-up maneuvers from down-down to a new equilibrium state



⇒ up-up



⇒ down-up

Experimental results

IEEE Robotics and Automation Letters, vol. 7(1), 2022 video



convergence in 2 iterations!

latest video with more simulations & experiments on YouTube https://youtu.be/1aKG__8gfvk